CYLAB SECURITY ACADEMY

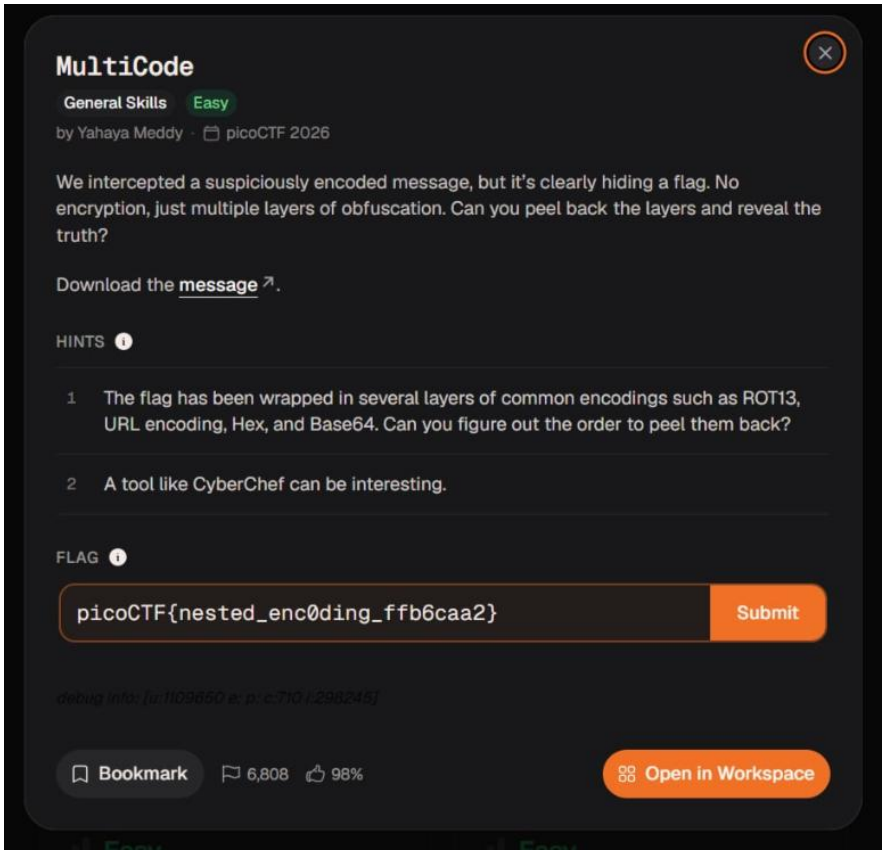
Challenge Write-Up Template

Challenge Information

Student Name	Keyzia Joy Pascua
Section / Class	BSIT 3 - Networking
Challenge Name	MultiCode
Category	General Skills
Points / Difficulty	Easy
Date Solved	August 11, 2026
Flag Format Given	picoCTF{...}

1. Challenge Description

This challenge is a good exercise in recognizing encoding fingerprints. You're given a single downloaded string that is a flag wrapped in several layers of obfuscation - not encryption - built from common encodings such as ROT13, URL encoding, Hex, and Base64. The task is to identify each layer by its distinctive pattern and peel them off one at a time, from the outside in, until the plain picoCTF flag is revealed.



2. Initial Analysis / Reconnaissance

The key skill here is pattern recognition: each encoding leaves a visual signature you can spot before decoding anything. A string made only of letters, digits, +, /, and = padding is almost always Base64 - that's the outermost layer in this case. Rather than guessing at every layer, decode the outermost one first, then re-examine the result: it will usually reveal a new, different-looking pattern that tells you what the next layer is.

3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
echo '...' | base64 -d	Decode the outermost Base64 layer of the downloaded message	A long hex-looking string of characters (0-9, a-f)
echo '...' | xxd -r -p	Convert the hex string back into raw text (hex decode)	cvpbPGS%7Barfgrq_rap0qvat_sso6pnn2%7D
python3 -c "import urllib.parse; print(urllib.parse.unquote('...'))"	URL-decode the %7B / %7D characters back into { }	cvpbPGS{arfgrq_rap0qvat_sso6pnn2}
echo '...' | tr 'A-Za-z' 'N-ZA-Mn-za-m'	Apply ROT13 to undo the letter-shift cipher	picoCTF{nested_enc0ding_ffb6caa2}

4. Solution Walkthrough

Step 1

Look at the raw downloaded string before doing anything else. If it ends in '=' padding and uses only Base64-safe characters (A-Z, a-z, 0-9, +, /), that's a strong signal the outer layer is Base64. Decode it with: echo '...' | base64 -d. The result here is a long string of only 0-9 and a-f characters - itself a fingerprint, this time for hex-encoded data.

```
kyxxajoy@PASCUA:~$ echo 'NjM3NjcwNjI1MDQ3NTMyNTM3NDI2MTcyNjY2NzcyNzE1ZjcyN
jE3MDMwNzE3NjYxNzQ1ZjczNzM2ZjM2NzA2ZTZlMzIyNTM3NDQ=' | base64 -d
6376706250475325374261726667727 15f726170307176174 5f73736f36706e6e32253744
```

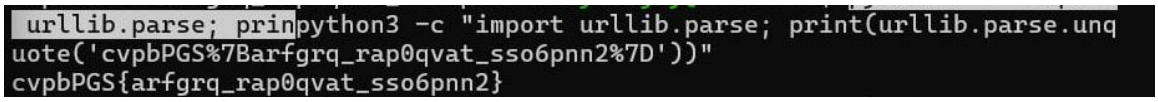
Step 2

When you see output made up purely of hex digits (0-9, a-f), convert it back to raw text with: echo '...' | xxd -r -p. In this case that produces cvpbPGS%7Barfgrq_rap0qvat_sso6pnn2%7D - readable-looking text, but with %7B and %7D sitting where you'd expect braces. Those %XX sequences are the signature of URL/percent-encoding.

```
kyxxajoy@PASCUA:~$ echo '6376706250475325374261726667727 15f726170307176617
45f73736f36706e6e32253744' | xxd -r -p
cvpbPGS%7Barfgrq_rap0qvat_sso6pnn2%7Dkyxxajoy@PASCUA:~$ python3 -c "import
```

Step 3

Whenever you see %XX sequences like %7B or %7D, decode them with a URL-decoding tool. Python's urllib.parse.unquote() works well for this: python3 -c "import urllib.parse; print(urllib.parse.unquote('...'))". Here it produces cvpbPGS{arfgrq_rap0qvat_sso6pnn2} - now shaped exactly like a flag, with braces in the right place, but the letters inside are still scrambled (cvpbPGS instead of picoCTF).



Additional steps (add as needed)

When the structure of a string looks correct but the letters themselves seem shifted or scrambled, suspect a Caesar-style cipher, and ROT13 is the most common one in CTFs. Reverse it with: echo '...' | tr 'A-Za-z' 'N-ZA-Mn-za-m'. Applying that here finally produces the real flag: picoCTF{nested_enc0ding_ffb6caa2}.

5. Flag

```
picoCTF{nested_enc0ding_ffb6caa2}
```

6. Key Concept / What I Learned

The main lesson is learning to recognize encoding fingerprints - Base64's charset and padding, pure hex digits, %XX URL escapes, and letter-shifted text - and peeling nested layers off in the right order, outside-in. It's also a reminder that obfuscation is not encryption: each layer here is trivially reversible once you can identify what produced it, which is why multi-layer 'encryption' challenges are usually just multi-step decoding exercises.

7. References

CyberChef is a useful tool for this kind of challenge - it can auto-detect and chain several of these encodings visually, which is a good way to double-check a layer order worked out by hand. Standard references on Base64, hex, URL encoding, and ROT13 cover the mechanics of each individual layer.